Syntax_3

Examples, test case for all (https://canvas.elte.hu/courses/65695/files/4479603/download?wrap=1)

Let's extend our "compiler" (actually at this point it is just a lexical+syntax analyzer) with the following new feature:

- ### Shift

  Extend the While language with a shift operator. The >> is a binary operator, taking two expressions as operands and yielding a more complex expression. It has the lowest precedence among the binary operators of While.

  ```
  a := 1 >> 2;
  ```

Ternary

Extend the While language with the following new operator. The so-called ternary or Elvis operator forms an expression from three sub-expressions, in the following way:
(*exp1* ? *exp2* : *exp3*), where each expN is an arbitrary expression.

```
a := (true?1:2);
```

To avoid any unambiguity a ternary expression is always parenthesized.

Parallelization

Introduce the **||** keyword into the lexical and syntax analysis phases to support the parallel execution of two statements. The **||** keyword is basically a binary operator on two statements, yielding a new statement.

Syntax: [*statement1* || *statement2*];

```
[a := b || b:=a];
```

Notice: be aware that there is no semicolon after *statement1* or *statement2*, just after the closing square-bracket.

Multiple declarations

Let's enhance the While language with a new feature to support multiple variable declarations at the same line, like this:

```
integer a, b, c;
```

Every variable should be separated by a comma, and the whole "line" should be closed by a semi-colon.

- ### Duplicate

  The "duplicate" keyword can be applied to any statement (but just statement, no expression, no declaration). As a result, the corresponding statement is evaluated two times. Modify the implementation of While's lexer&parser in a way to support this new construct.

  It is important to note, that a 'duplicate' keyword does not form a statement in itself, it is just a modifier to an already existing statement (forming altogether a new statement).

  ```
  duplicate write(a);
  ```

- ### List

  Let's introduce typed lists into the While language. A declaration of a list type always looks like this <element-type> "list" keyword followed by the name of the variable:

  ```
  integer list a;
  ```

  The second element of the 'a' list can be accessed by the a[2] expression. This indexing expression can be used the get and also set the element on the given index. The index can only be a number literal, no complex expression is allowed!

  A list has a form of elements separated by commas and this all is enclosed by square brackets:

  ```
  [1, a, 2, 3, b]
  ```

  A list in-itself is an expression, therefore it can be used on the right-hand-side of an assignment.

  A new "++" binary operator should be added to support the concatenation of two lists, like this:

  ```
  b := [1,2,3,4,5,6,7];
  a := b ++ [100,10,1]
  ```

  There is another special construct introduced, the += operator, which is a special kind of assignment (to be more precise it is an "appendment"). It takes a variable on the left-hand side and a list on the right-hand side, and appends them.

  ```
  a += [2, 4, 6, 8, 10, 100];
  ```

- ### Tuple

  Add support for tuples in While. With tuples, you can basically pack types together to form a new type. This new type can be thought of as a record type where fields are not identified by their name just their index.

A tuple declaration looks like this:

```
<integer, boolean, integer> a;
<integer, integer> b;
```

In this case, the 'a' variable can store 2 integer and 1 boolean values, these values can be referenced in the following way:

```
a.1
a.2
a.3
```

These field indexes can be used to get and also to set the value of that specific field. The field indexes can only be number literals, no complex expression is allowed!

- **Simultan**

  Add simultaneous assignments to the language. That is, an assignment statement can change multiple variables at once. Left-values (identifier names) and right-values are separated by commas. Note that the number of variables changed simultaneously is not limited. An error should be reported if there is a different number of left-values than right-values.

  For example:

```
x,y := 1+2, true;
```

```
x,y,z := 1+2, true, 3;
```